AUTOLEASE PERÚ

Base de Datos

Avance de proyecto – Sesión 09: diseño e implementación de la base de datos de leasing vehicular

Integrantes: _____ · Docente: María Haydee Li Fernandez · III Ciclo 2026-2

RUTA DE LA PRESENTACIÓN

Vamos paso a paso: de la idea del negocio hasta las consultas en SQL Server

1 **Empresa** → 2 **Problema** → 3 **Procesos** → 4 **Objetivos**

5 **Reglas** → 6 **Conceptual** → 7 **Lógico** → 8 **Físico**

Fase de negocio (1-5) → Fase de diseño de datos (6-7) → Fase de implementación en SQL Server (8)

1. PRESENTACIÓN DE LA EMPRESA

AutoLease Perú S.A.C. — empresa de leasing y financiamiento vehicular

1 ¿Quiénes somos?

- Empresa peruana de leasing (arrendamiento financiero) de vehículos.
- Atendemos a personas naturales y empresas.

2 ¿Qué ofrecemos?

- Contrato a 3 años con cuota mensual fija.
- Seguro, mantenimiento programado y trámite de matrícula incluidos.

3 ¿Dónde estamos?

- Sede San Isidro (Lima)
- Sede Miraflores (Lima)
- Sede Arequipa

Actividad económica: servicios financieros — arrendamiento y financiamiento de vehículos.

2. REALIDAD PROBLEMÁTICA

ANTES (sin base de datos) → DESPUÉS (con base de datos)

ANTES

✗ Excel y papel con datos duplicados

✗ Morosidad detectada tarde

✗ Seguros y mantenimientos sin trazabilidad

✗ No se sabe qué vehículo está disponible

✗ Reportes por sucursal toman horas

DESPUÉS

✓ Información centralizada en SQL Server

✓ Pagos y atrasos consultables al instante

✓ Seguro y mantenimiento ligados a cada vehículo

✓ Disponibilidad de vehículos en una consulta

✓ Reportes con un SELECT

3. IDENTIFICACIÓN DE PROCESOS

Mapa de procesos que cubre la base de datos

1

Registro de clientes

Se guardan datos, canal de captación y cónyuge.

Tablas involucradas

cliente

2

Gestión de contratos

Un empleado asigna un vehículo y fija cuota y fechas.

Tablas involucradas

contrato_leasing · vehiculo · empleado

3

Control de pagos

Cada contrato genera cuotas mensuales con estado.

Tablas involucradas

pago

4

Seguro y matrícula

Cada vehículo tiene seguro; cada contrato, placa.

Tablas involucradas

seguro · matricula

5

Mantenimiento

Proveedores atienden los vehículos; se registra el costo.

Tablas involucradas

mantenimiento · proveedor

El flujo va de izquierda a derecha: captar → contratar → cobrar → asegurar/matricular → mantener.

4. OBJETIVOS

4.1 OBJETIVO GENERAL

Diseñar e implementar una base de datos relacional en SQL Server que centralice y controle el proceso de leasing vehicular de AutoLease Perú.

4.2 OBJETIVOS ESPECÍFICOS

1. **Modelar entidades y relaciones (DER)**
2. **Normalizar hasta 3FN**
3. **Crear tablas con claves y restricciones**
4. **Cargar 3 meses de datos y consultarlos**
5. **Obtener reportes de pagos y morosidad**

5. REGLAS DEL NEGOCIO

Cada regla se convierte en una relación o restricción de la base de datos

R1 — **Sucursal 1—N Empleado**

→ FK sucursal_id en empleado

R2 — **Cliente 1—N Contrato**

→ FK cliente_id en contrato_leasing

R3 — **Cliente casado con Cliente (0..1)**

→ FK conyuge_id en cliente

R4 — **Empleado 1—N Contrato**

→ FK empleado_id en contrato_leasing

R5 — **Contrato 1—1 Vehículo / Vehículo 1—1 Seguro**

→ FK + UNIQUE

R6 — **Contrato 1—N Pago**

→ FK contrato_id en pago

R7 — **Contrato 1—1 Matrícula**

→ FK + UNIQUE contrato_id

R8 — **Cuota > 0 y fecha_fin > fecha_inicio**

→ CHECK

6. DISEÑO CONCEPTUAL

Primero entendemos 3 conceptos clave del modelo entidad-relación

1 ENTIDAD

Algo del negocio sobre lo que guardamos datos.

Ej.: Cliente, Vehículo

se vuelve TABLA

2 ATRIBUTO

Una característica de la entidad.

Ej.: nombre, precio, color

se vuelve COLUMNA

3 RELACIÓN

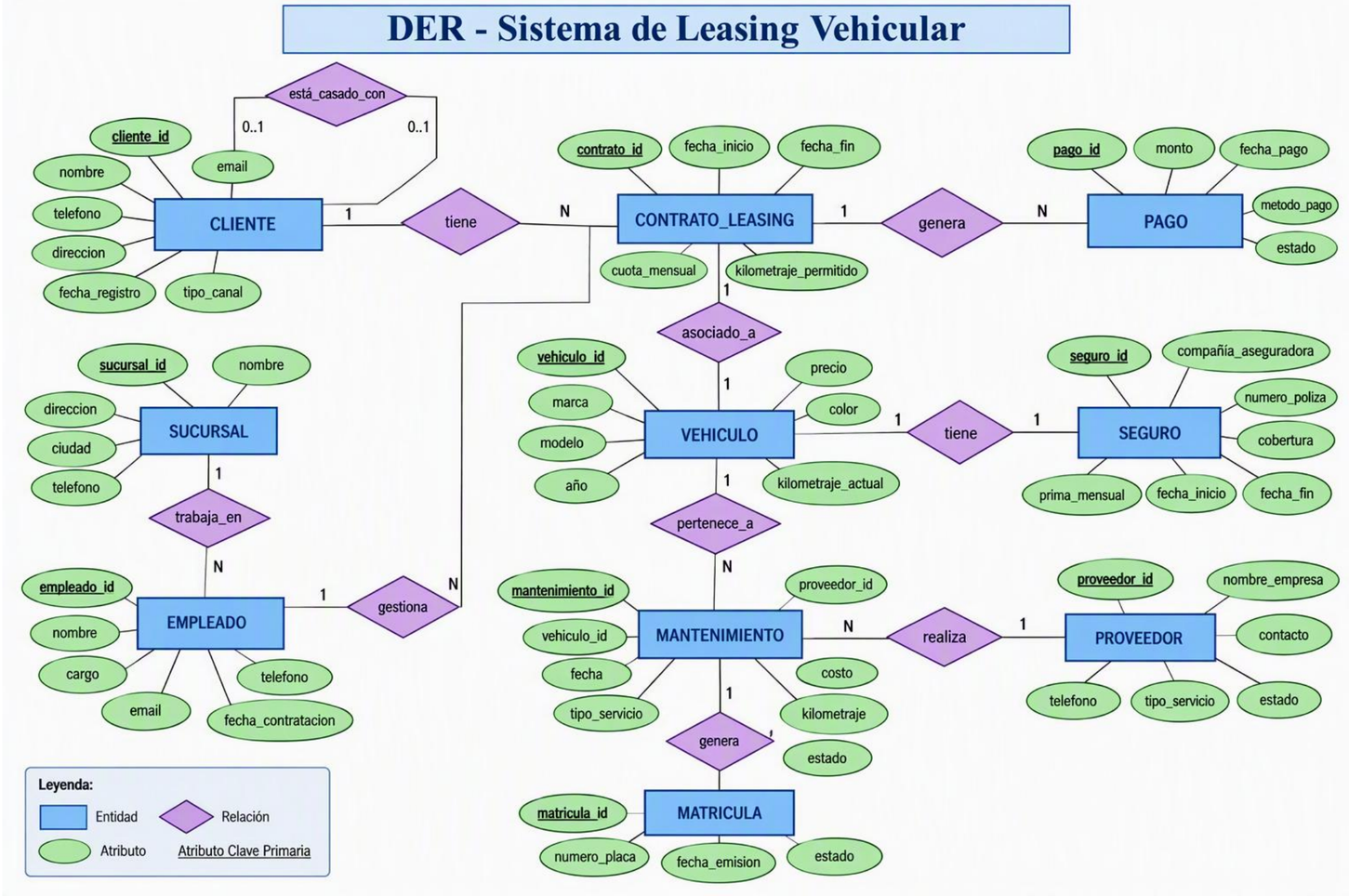
Cómo se conectan dos entidades.

Ej.: Cliente 'tiene' Contrato

se vuelve LLAVE FORÁNEA

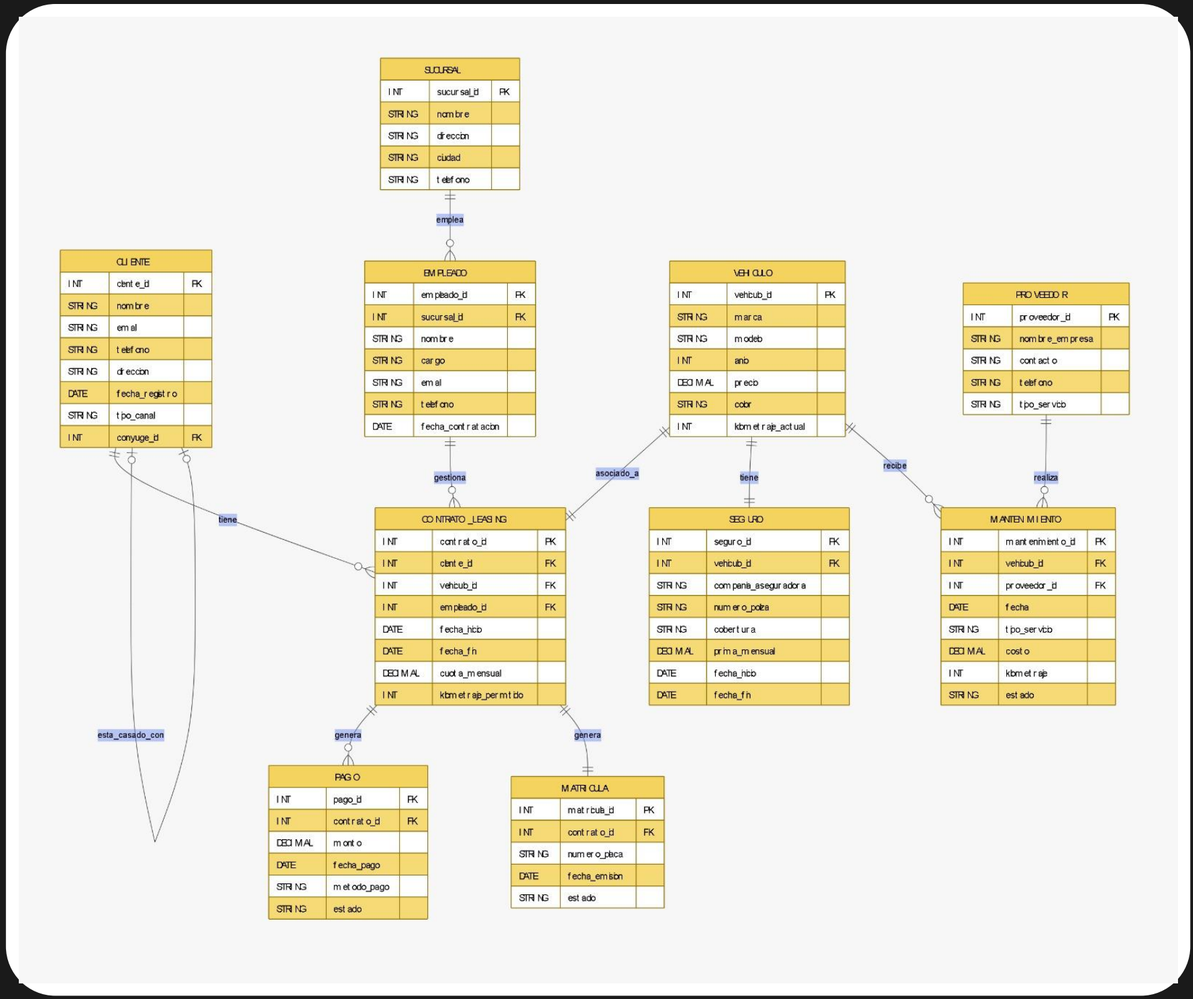
6.1 DIAGRAMA ENTIDAD-RELACIÓN

Lectura: 1 cliente → N contratos · 1 contrato → 1 vehículo, 1 matrícula y N pagos



7. DISEÑO LÓGICO

7.1 Diagrama relacional: cada entidad ya es una tabla con PK y FK



Cómo leerlo

PK — Llave primaria: identifica cada fila (única).

FK — Llave foránea: apunta a la PK de otra tabla.

1:N — Un registro padre, muchos hijos.

1:1 — Se logra con FK + UNIQUE.

7.1 NORMALIZACIÓN

Cada forma normal arregla un tipo de problema distinto

	Regla	✗ ANTES	✓ DESPUÉS
1FN	**Un dato por celda**	pago → monto: '520, 520, 520'	**3 filas en pago, 1 monto cada una**
2FN	**Todo depende de TODA la clave**	(contrato, vehículo) + precio del vehículo	**precio se guarda en vehiculo (PK simple)**
3FN	**Nada depende de otro dato no clave**	empleado guarda sucursal_id y ciudad	**ciudad vive solo en sucursal**

7.2 LAS 10 TABLAS DEL MODELO

PK = llave primaria · FK = llave foránea

1

sucursal

PK sucursal_id

2

empleado

PK · FK sucursal

3

cliente

PK · FK conyuge

4

vehiculo

PK vehiculo_id

5

proveedor

PK proveedor_id

6

contrato_leasing

PK · FK cliente, vehículo, empleado

7

seguro

PK · FK vehículo (única)

8

mantenimiento

PK · FK vehículo, proveedor

9

pago

PK · FK contrato

10

matricula

PK · FK contrato (única)

8. DISEÑO FÍSICO

Pasamos del dibujo a SQL Server: dos familias de comandos

8.1 DDL

Define la ESTRUCTURA

- **CREATE → construir**
- **ALTER → modificar**
- **TRUNCATE → vaciar**
- **DROP → eliminar**

8.2 DML

Manipula los DATOS

- **INSERT → agregar**
- **UPDATE → cambiar**
- **DELETE → borrar**
- **SELECT → consultar**

8.1.1 CREATE: LA BASE DE DATOS

Primero creamos el 'contenedor' y luego le decimos a SQL Server que lo use

```
CREATE DATABASE db_leasing_vehicular;
GO

-- Seleccionar la BD de trabajo
USE db_leasing_vehicular;
GO
```

1 **CREATE DATABASE**
Crea la base de datos vacía.

2 **GO**
Separa lotes: SSMS ejecuta lo anterior antes de seguir.

3 **USE**
Todo lo que sigue se crea dentro de esta BD.

💡 Tip de SSMS

Selecciona el bloque y presiona F5.

Si ejecutas todo junto, verifica que 'db_leasing_vehicular' aparezca en el Explorador de objetos (clic derecho → Actualizar).

8.1.1 CREATE TABLE: ANATOMÍA DE UNA TABLA

Ejemplo con la tabla empleado

```sql
CREATE TABLE empleado (
  empleado_id INT IDENTITY(1,1) NOT NULL,
  sucursal_id INT NOT NULL,
  nombre VARCHAR(80) NOT NULL,
  cargo VARCHAR(40) NOT NULL,
  email VARCHAR(80) NULL,
  telefono VARCHAR(15) NULL,
  fecha_contratacion DATE NOT NULL,
  CONSTRAINT pk_empleado
    PRIMARY KEY (empleado_id),
  CONSTRAINT fk_empleado_sucursal
    FOREIGN KEY (sucursal_id)
    REFERENCES sucursal(sucursal_id)
);
```

1. **IDENTITY(1,1)**
   ID autonumérico: empieza en 1 y sube de 1 en 1.

2. **NOT NULL / NULL**
   Obligatorio u opcional.

3. **PRIMARY KEY**
   Identifica cada empleado de forma única.

4. **FOREIGN KEY**
   Obliga a que la sucursal exista en la tabla sucursal.

8.1.1 RELACIONES ESPECIALES

Cómo se construyen el 1:1 y la autorrelación

1 Relación 1:1

Un vehículo tiene UN seguro.

UNIQUE impide repetir el vehiculo_id.

```
CREATE TABLE seguro (
  ...
  CONSTRAINT uq_seguro_vehiculo
    UNIQUE (vehiculo_id),
  CONSTRAINT fk_seguro_vehiculo
    FOREIGN KEY (vehiculo_id)
    REFERENCES vehiculo(vehiculo_id)
);
```

2 Autorrelación

Un cliente puede estar casado con otro cliente.

La FK apunta a la MISMA tabla.

```
CREATE TABLE cliente (
  ...
  conyuge_id INT NULL,
  CONSTRAINT fk_cliente_conyuge
    FOREIGN KEY (conyuge_id)
    REFERENCES cliente(cliente_id)
);
```

8.1.3 ALTER: MODIFICAR SIN VOLVER A CREAR

Seis operaciones, seis ejemplos

1 Agregar columna

```
ALTER TABLE cliente
  ADD dni VARCHAR(8) NULL;
```

2 Cambiar columna

```
ALTER TABLE cliente
  ALTER COLUMN dni CHAR(8) NOT
NULL;
```

3 Agregar UNIQUE

```
ALTER TABLE cliente
  ADD CONSTRAINT uq_cliente_dni
  UNIQUE (dni);
```

4 Agregar CHECK

```
ALTER TABLE vehiculo
  ADD CONSTRAINT ck_vehiculo_precio
  CHECK (precio > 0);
```

5 Agregar DEFAULT

```
ALTER TABLE pago
  ADD CONSTRAINT df_pago_estado
  DEFAULT 'Pendiente' FOR estado;
```

6 Quitar columna / restricción

```
ALTER TABLE tabla_prueba
  DROP CONSTRAINT
fk_prueba_sucursal;
ALTER TABLE tabla_prueba
  DROP COLUMN descripcion;
```

DELETE VS TRUNCATE VS DROP

¿Cuál uso para eliminar? Depende de QUÉ quiero eliminar

DELETE

¿Qué elimina?: Borra filas elegidas

Condición: WHERE

Estructura: Conserva

Detalle: Sí (en transacción)

Tipo: DML

TRUNCATE

¿Qué elimina?: Borra TODAS las filas

Condición: No usa WHERE

Estructura: Conserva

Detalle: Reinicia IDENTITY

Tipo: DDL

DROP

¿Qué elimina?: Borra la tabla completa

Condición: —

Estructura: Elimina

Detalle: No hay vuelta atrás

Tipo: DDL

```
DELETE FROM proveedor WHERE nombre_empresa = 'Proveedor Prueba SAC';   -- una fila
TRUNCATE TABLE tabla_prueba;                                            -- todas las filas
DROP TABLE tabla_prueba;                                                -- tabla entera
```

8.1.2 TRUNCATE Y 8.1.4 DROP EN ACCIÓN

Paso a paso sobre una tabla de prueba

1 Crear datos de prueba

```
INSERT INTO tabla_prueba (id,
sucursal_id)
VALUES (1,1), (2,2), (3,3);
```

Resultado: 3 filas

2 TRUNCATE

```
TRUNCATE TABLE tabla_prueba;
SELECT * FROM tabla_prueba;
```

Resultado: 0 filas · la tabla sigue existiendo

3 DROP

```
DROP TABLE tabla_prueba;
```

Resultado: La tabla ya no existe

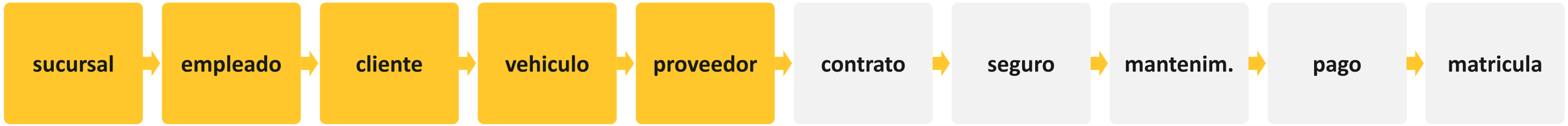
8.2.1 INSERT: AGREGAR DATOS

Regla de oro: se insertan primero las tablas 'padre' y luego las 'hijas'

```
INSERT INTO sucursal
  (nombre, direccion, ciudad, telefono)
VALUES
 ('Sede San Isidro','Av. Javier Prado 1250',
  'Lima','014567890'),
 ('Sede Arequipa','Av. Ejercito 450',
  'Arequipa','054223344');
```

1. **INSERT INTO tabla (columnas)**
   Indica dónde y qué columnas llenar.
2. **VALUES (...)**
   Un paréntesis por fila; mismo orden que las columnas.
3. **IDENTITY**
   sucursal_id no se escribe: SQL Server lo genera.

Orden de carga (de padres a hijas):



Datos cargados: junio – septiembre 2026 (3 meses) · 10 clientes · 8 contratos · 18 pagos

8.2.2 DELETE Y 8.2.3 UPDATE

⚠ Sin WHERE afectas TODAS las filas — siempre filtra

```sql
UPDATE pago
SET estado = 'Pagado'
WHERE contrato_id = 3
  AND fecha_pago = '2026-09-10';
```

```sql
DELETE FROM proveedor
WHERE nombre_empresa =
   'Proveedor Prueba SAC';
```

Antes → Después

contrato	fecha_pago	estado
3	2026-09-10	Pendiente



contrato	fecha_pago	estado
3	2026-09-10	Pagado

1. **UPDATE ... SET**
   Cambia el valor de una o más columnas.

2. **DELETE FROM**
   Elimina las filas que cumplan el WHERE.

3. **WHERE**
   Es el filtro de seguridad: sin él se modifica toda la tabla.

8.2.4 SELECT: ANATOMÍA DE UNA CONSULTA

Se escribe en un orden y se piensa en otro

```sql
SELECT vehiculo_id, marca,
       modelo, precio
FROM vehiculo
WHERE anio = 2026
  AND precio > 25000
ORDER BY precio DESC;
```

1 FROM
¿De qué tabla?

2 WHERE
¿Qué filas?

3 SELECT
¿Qué columnas?

4 ORDER BY
¿En qué orden?

Resultado en SSMS

id	marca	modelo	precio
4	Toyota	RAV4	34500.00
8	Chevrolet	Tracker	29900.00
2	Hyundai	Tucson	28900.00

Operadores usados

= y > → relacionales

AND → lógico

precio * 1.18 → aritmético

SUBCONSULTAS: UNA CONSULTA DENTRO DE OTRA

Se resuelve primero la de adentro y su resultado alimenta a la de afuera

Mono-registro

Devuelve UN valor

```sql
SELECT marca, modelo, precio
FROM vehiculo
WHERE precio =
 (SELECT MAX(precio)
  FROM vehiculo);
```

1° MAX(precio) = 34500

2° vehículo con ese precio → RAV4

Multi-registro

Devuelve VARIOS valores

```sql
SELECT cliente_id, nombre
FROM cliente
WHERE cliente_id IN
 (SELECT cliente_id
  FROM contrato_leasing);
```

1° lista de clientes con contrato

2° clientes que están en la lista

Multi-columna

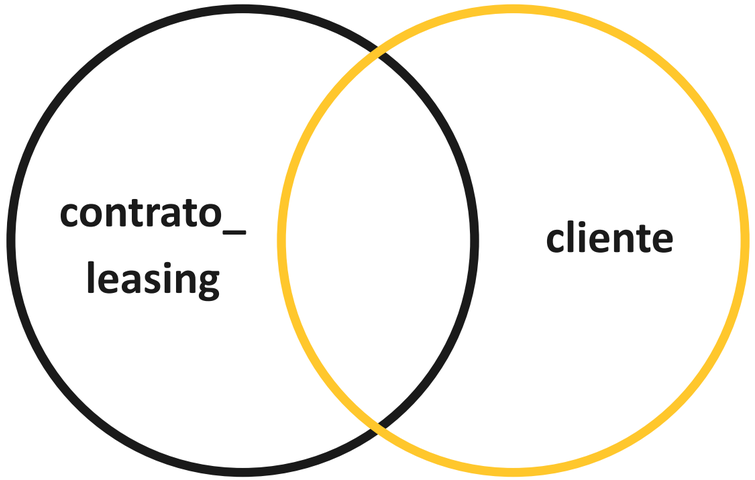
Compara VARIAS columnas

```sql
SELECT p.contrato_id,
       p.fecha_pago, p.monto
FROM pago p
WHERE p.fecha_pago =
 (SELECT MAX(p2.fecha_pago)
  FROM pago p2 WHERE
  p2.contrato_id=p.contrato_id);
```

Obtiene el ÚLTIMO pago de cada contrato

8.2.4.5 INNER JOIN: UNIR TABLAS

Devuelve solo las filas que coinciden en AMBAS tablas



∩

ON c.cliente_id = cl.cliente_id

(la intersección)

```sql
SELECT c.contrato_id,
       cl.nombre AS cliente,
       v.marca, v.modelo, c.cuota_mensual
FROM contrato_leasing c
INNER JOIN cliente cl
   ON c.cliente_id = cl.cliente_id
INNER JOIN vehiculo v
   ON c.vehiculo_id = v.vehiculo_id;
```

contrato	cliente	marca	modelo	cuota
1	Carlos Gutierrez Perez	Toyota	Corolla	520.00
2	Pedro Villanueva Leon	Hyundai	Tucson	610.00
3	Lucia Campos Rios	Kia	Rio	480.00

Cada INNER JOIN agrega una tabla; ON indica qué FK las une.

8.2.5 FUNCIONES DE COLUMNA

Resumen de muchas filas en un solo valor

COUNT(*)	COUNT(DISTINCT)	SUM()	AVG()	MIN() / MAX()
Cuenta filas	Cuenta sin repetir	Suma	Promedio	Mínimo y máximo
8 contratos	**3 canales**	**total recaudado**	**precio 26,100**	**430 / 820**

```sql
SELECT contrato_id, SUM(monto) AS total_pagado
FROM pago
WHERE estado = 'Pagado'
GROUP BY contrato_id
HAVING SUM(monto) > 1000;
```

1. **GROUP BY**
   Agrupa las filas por contrato.
2. **SUM**
   Suma los pagos de cada grupo.
3. **HAVING**
   Filtra grupos (WHERE filtra filas).

8.2.6 FUNCIONES DE FECHA

Para calcular plazos, vencimientos y reportes por mes

Función	¿Qué hace?	Ejemplo en el proyecto	Resultado
GETDATE()	Fecha y hora actual	SELECT GETDATE()	fecha de hoy
YEAR / MONTH / DAY	Extrae una parte de la fecha	MONTH(fecha_inicio)	7, 8, 9...
DATEDIFF	Diferencia entre fechas	DATEDIFF(MONTH, fecha_inicio, fecha_fin)	36 meses
DATEADD	Suma tiempo a una fecha	DATEADD(MONTH, 1, fecha_inicio)	1 mes después
DATENAME	Nombre del mes/día	DATENAME(MONTH, fecha_pago)	July, August...
FORMAT	Da formato	FORMAT(fecha_pago,'dd/MM/yyyy')	01/07/2026

Uso real: contar pagos por mes → GROUP BY DATENAME(MONTH, fecha_pago) para ver la recaudación mensual de jun–sep.

CONCLUSIONES Y SIGUIENTES PASOS

LO LOGRADO

- ✓ Modelo de 10 tablas en 3FN
- ✓ DDL con claves, UNIQUE, CHECK y DEFAULT
- ✓ 3 meses de datos cargados
- ✓ Consultas: simples, subconsultas, JOIN, funciones

ENTREGA FINAL

1. 8.3 DCL: GRANT, REVOKE, DENY
2. 8.4 Procedimientos almacenados (mín. 4)
3. 8.5 Vistas
4. 8.6 Conectividad · 8.7 Seguridad